

Getting Comfortable with Postgres

Tools, Schemas, Users, Objects, Arrays, JSONB & Window Functions

Session 1 of 3 — Beginning Postgres Workshop

PGSummit NYC 2026

Session 1 Topics

- Tools for querying Postgres
- Schemas
- Users, roles & permissions
- Data types, constraints & building objects (with a live demo)
- Arrays and JSON
- Window functions and CTEs

Tools for Querying Postgres

GUI Tools

pgAdmin

- Official GUI tool
- Web-based interface
- Free and open source

DBeaver

- Multi-database support
- ER diagrams
- Free community edition

psql - The Postgres CLI

The most powerful way to interact with Postgres

```
psql (18.x)
```

```
Type "help" for help.
```

```
bluebox=#
```

Essential psql Commands

Command	Description
<code>\l</code>	List all databases
<code>\c dbname</code>	Connect to database
<code>\dt</code>	List tables
<code>\d tablename</code>	Describe table
<code>\dn</code>	List schemas

- `\d` stands for "describe"

Expanded Display: \x auto

```
\x auto    -- Let psql decide (recommended!)
\x on      -- Always expanded
\x off     -- Always horizontal (default)
```

Expanded view shows one column per line:

```
-[ RECORD 1 ]-----
id          | 1
name        | Alice
email       | alice@test.com
```

Pro tip: Put `\x auto` in your `~/.psqlrc` file!

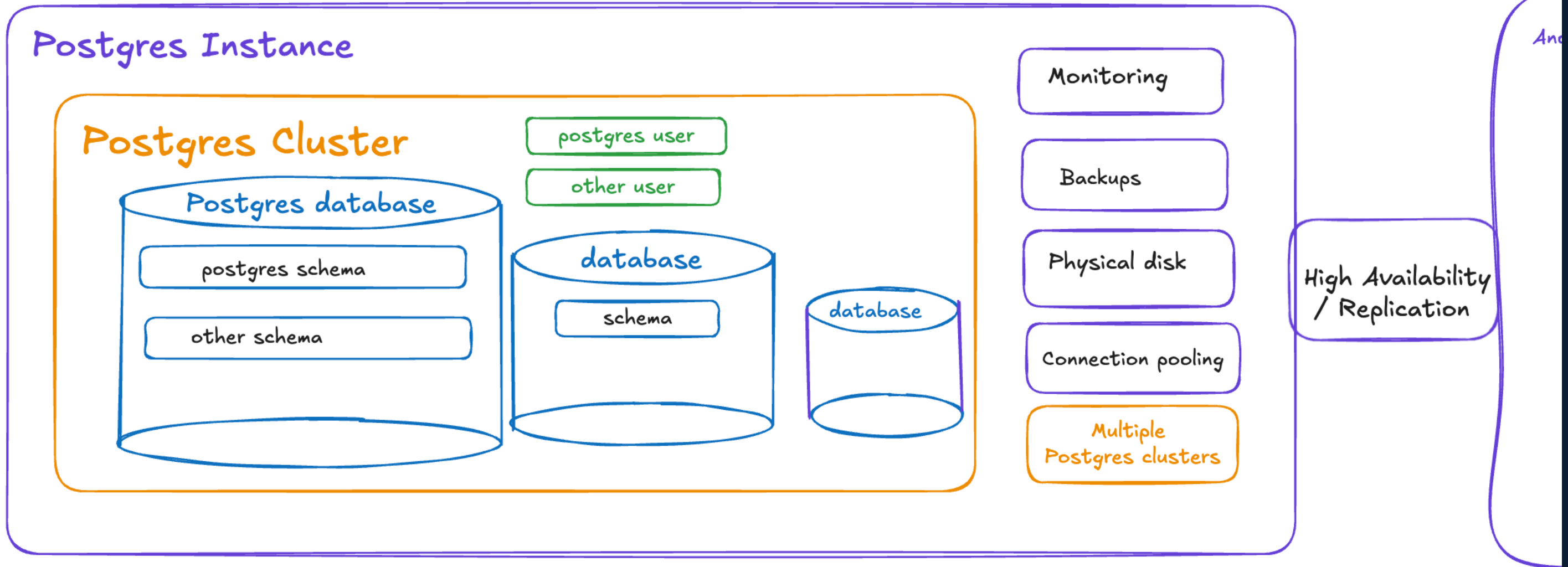
More Useful psql Settings

```
\timing          -- Show query execution time
\conninfo        -- Show current connection info
\pset pager off  -- Turn off page-by-page scrolling
```

```
bluebox=# \timing
Timing is on.
bluebox=# SELECT COUNT(*) FROM bluebox.film;
 count
-----
    7836
(1 row)
Time: 2.145 ms
```

```
bluebox=# \conninfo
You are connected to database "bluebox" as user "postgres"
on host "localhost" at port "5432".
```

Schemas



What is a Schema?

A **schema** is a namespace within a database

- Organizes database objects (tables, views, functions)
- Provides access control boundaries
- Avoids naming conflicts

Default Schema

-- The default schema is 'public'

```
CREATE TABLE my_table (id int);
```

-- Same as:

```
CREATE TABLE public.my_table (id int);
```

Creating Schemas

-- Bluebox already has its schema, but you could create more:
`CREATE SCHEMA reporting;`

-- Then create tables in that schema
`CREATE TABLE reporting.daily_stats (...);`

Bluebox uses `bluebox` schema to organize all its tables

Schema Search Path

-- View current search path

```
SHOW search_path;
```

-- Output: "\$user", public

-- Set search path

```
SET search_path TO bluebox, public;
```

-- Now queries will look in bluebox first

```
SELECT * FROM film; -- Same as bluebox.film
```

Bluebox Schema

The Bluebox database uses a bluebox schema with 17 tables:

```
SET search_path TO bluebox, public;  
\dt
```

Schema	tablename	
bluebox	customer	(186,740 rows)
bluebox	film	(7,836 rows)
bluebox	person	(258,772 rows)
bluebox	store	(196 rows)

...and 13 more tables

Bluebox Tables

\d bluebox.film

Column	Type	Description
film_id	bigint	Primary key
title	text	Movie title
overview	text	Plot summary
release_date	date	Release date
vote_average	real	TMDB rating (0-10)
popularity	real	TMDB popularity score
budget	bigint	Production budget
revenue	bigint	Box office revenue

Users and Permissions

Postgres Roles

In Postgres, **users** and **groups** are both **roles**

-- Create a user (role with login)

```
CREATE ROLE app_user WITH LOGIN PASSWORD 'secure_password';
```

-- Create a group (role without login)

```
CREATE ROLE readonly;
```

Role Attributes

```
CREATE ROLE app_user WITH  
  LOGIN  
  PASSWORD 'secure_password'  
  CREATEDB  
  CREATEROLE;
```

Common attributes:

- LOGIN / NOLOGIN
- SUPERUSER / NOSUPERUSER
- CREATEDB / NOCREATEDB
- CREATEROLE / NOCREATEROLE

Granting Privileges

-- Grant connection to database

```
GRANT CONNECT ON DATABASE bluebox TO app_user;
```

-- Grant schema usage

```
GRANT USAGE ON SCHEMA bluebox TO app_user;
```

-- Grant table privileges

```
GRANT SELECT ON ALL TABLES IN SCHEMA bluebox TO app_user;
```

-- Grant specific privileges

```
GRANT SELECT, INSERT, UPDATE ON bluebox.rental TO app_user;
```

Manage Privileges with Groups

Roles can be members of other roles (like groups!)

-- Create a group role for data analytics team

```
CREATE ROLE data_analytics NOLOGIN;
```

-- Grant read access on Bluebox to the group

```
GRANT CONNECT ON DATABASE bluebox TO data_analytics;
```

```
GRANT USAGE ON SCHEMA bluebox TO data_analytics;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA bluebox TO data_analytics;
```

Grant Users Role Membership

-- Create a user for an analyst

```
CREATE ROLE maria LOGIN;
```

```
\password maria
```

-- Add maria to the data_analytics group

```
GRANT data_analytics TO maria;
```

-- Maria now inherits all permissions from data_analytics!

View Role Memberships

\du

List of roles		
Role name	Attributes	Member of
data_analytics	Cannot login	{}
maria		{data_analytics}
app_user		{}
postgres	Superuser, Create role, ...	{}

Maria is a member of data_analytics - she inherits its permissions!

ALTER User Search Path

-- Set search path

```
ALTER USER maria SET search_path TO bluebox, public;
```

-- From next login, queries will look in bluebox first

```
SELECT * FROM film; -- Same as bluebox.film
```

Object and Data Types

Database Objects

- **Tables** - Store data in rows and columns
- **Views** - Saved queries that act like tables
- **Indexes** - Speed up data retrieval
- **Sequences** - Auto-incrementing number generators
- **Functions** - Reusable SQL/procedural code

Common Data Types

Numeric

- INTEGER / BIGINT
- NUMERIC(p, s)
- REAL / DOUBLE

Date/Time

- DATE / TIME
- TIMESTAMP
- TIMESTAMPTZ ★
- INTERVAL

Character

- TEXT (preferred)
- VARCHAR(n)

Other

- BOOLEAN
- UUID
- JSON / JSONB
- array of any other type, e.g. int[], text[]



Time: Use **TIMESTAMPTZ**!

TIMESTAMP vs TIMESTAMPTZ - always prefer **TIMESTAMPTZ**

-- TIMESTAMP: No timezone info (dangerous!)

```
CREATE TABLE events (event_time TIMESTAMP);
```

-- TIMESTAMPTZ: Stores in UTC, converts on display ✓

```
CREATE TABLE events (event_time TIMESTAMPTZ);
```

-- What time is it?

```
SELECT NOW(); -- 2026-01-16 10:30:00-08
```

TIMESTAMPTZ handles daylight saving automatically!

Use NUMERIC, Not MONEY!

```
-- DON'T use the MONEY type
price MONEY -- ❌ Locale-dependent, rounding issues

-- DO use NUMERIC for currency
price NUMERIC(10,2) -- ✓ Exact precision, no surprises
```

Bluebox uses NUMERIC for payment amounts:

```
SELECT amount FROM bluebox.payment LIMIT 3;
```

```
amount
-----
1.99
1.99
3.98
```

NUMERIC stores exact values - no floating point errors!

Custom Data Types: ENUM

Postgres lets you create custom types!

Bluebox uses an ENUM for MPAA ratings:

```
CREATE TYPE mpaa_rating AS ENUM (  
    'G', 'PG', 'PG-13', 'R', 'NC-17', 'NR'  
);
```

```
-- Used in the film table  
SELECT title, rating FROM bluebox.film  
WHERE rating = 'PG-13' LIMIT 3;
```

title	rating
Are You There God? It's Me, Margaret.	PG-13
Love at First Sight	PG-13
Batman Returns	PG-13

Constraints

Rules that enforce data integrity at the database level

Constraint	Purpose
PRIMARY KEY	Unique identifier for each row
FOREIGN KEY	Links to another table's primary key (REFERENCES)
NOT NULL	Column must have a value
UNIQUE	No duplicate values allowed
CHECK	Custom validation rules
DEFAULT	Auto-fill value if none provided

The table we're about to create uses four of these.

Creating a Table

Let's add a customer reviews feature to Bluebox!

```
CREATE TABLE bluebox.customer_review (  
    review_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES bluebox.customer(customer_id),  
    film_id BIGINT REFERENCES bluebox.film(film_id),  
    rating SMALLINT CHECK (rating BETWEEN 1 AND 5),  
    review_text TEXT,  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);
```



Demo: Add Yourself as a Customer

```
-- Add yourself to the Bluebox database
INSERT INTO bluebox.customer
  (customer_id, store_id, full_name, email)
VALUES (
  (SELECT MAX(customer_id) + 1 FROM bluebox.customer),
  1,                                -- Store #1
  'Your Name Here',                -- Your name!
  'you@example.com'
)
RETURNING customer_id, full_name;
```

customer_id		full_name
205026		Your Name Here

← Save this ID!

Note: this is a great use of the Postgres feature RETURNING



Demo: Find Films to Review

```
-- Find some popular films to review
SELECT film_id, title FROM bluebox.film
WHERE title IN ('The Dark Knight', 'Inception',
               'Interstellar', 'Dune: Part Two');
```

film_id		title
155		The Dark Knight
27205		Inception
157336		Interstellar
693134		Dune: Part Two



Demo: Insert Your Reviews

Replace 205026 with your actual customer_id!

-- Use YOUR customer_id from the previous step!

```
INSERT INTO bluebox.customer_review
    (customer_id, film_id, rating, review_text)
VALUES
    (205026, 155, 5, 'Heath Ledger was incredible!'),
    (205026, 27205, 5, 'Mind-bending! Had to watch twice'),
    (205026, 693134, 5, 'Even better than the first Dune!');
```

Joining Tables

Reviews store IDs; a **JOIN** matches rows across tables to get names back

```
SELECT f.title, p.name as actor
FROM bluebox.film f
INNER JOIN bluebox.film_cast fc ON f.film_id = fc.film_id
INNER JOIN bluebox.person p ON fc.person_id = p.person_id
WHERE f.title = 'The Dark Knight'
LIMIT 3;
```

title	actor
The Dark Knight	Gary Oldman
The Dark Knight	Morgan Freeman
The Dark Knight	William Fichtner

INNER JOIN keeps matching rows only; **LEFT JOIN** also keeps rows with no match

Demo: Querying Your Reviews

-- See your reviews with a JOIN

```
SELECT c.full_name, f.title, r.rating,  
       LEFT(r.review_text, 30) as review  
FROM bluebox.customer_review r  
JOIN bluebox.customer c ON r.customer_id = c.customer_id  
JOIN bluebox.film f ON r.film_id = f.film_id  
ORDER BY r.created_at DESC;
```

full_name	title	rating	review
Your Name	The Dark Knight	5	Heath Ledger was incredible
Your Name	Inception	5	Mind-bending! Had to watch
Your Name	Dune: Part Two	5	Even better than the first

Arrays and JSON

Arrays in PostgreSQL

Bluebox stores genre IDs as an array on each film:

```
SELECT film_id, title, genre_ids
FROM bluebox.film
WHERE title = 'The Dark Knight';
```

film_id	title	genre_ids
155	The Dark Knight	{18,28,80,53}

-- What do these IDs mean? Check the film_genre table

```
SELECT genre_id, name FROM bluebox.film_genre
WHERE genre_id IN (18, 28, 80, 53);
```

Drama (18), Action (28), Crime (80), Thriller (53)

Adding to an Array

```
BEGIN;  -- demo only: we'll roll this back

-- Add a genre to a film's array
UPDATE bluebox.film
SET genre_ids = genre_ids || ARRAY[9648]  -- Add Mystery
WHERE film_id = 155
RETURNING genre_ids;
-- {18,28,80,53,9648}

-- Alternative: array_append(genre_ids, 9648)

ROLLBACK;  -- keep The Dark Knight's genres unchanged
```

Array Operators

Operator	Meaning	Example
<code>= ANY ()</code>	Contains value	<code>28 = ANY(genre_ids)</code>
<code>@></code>	Contains all	<code>genre_ids @> ARRAY[28,80]</code>
<code><@</code>	Is contained by	<code>ARRAY[28] <@ genre_ids</code>
<code>&&</code>	Overlaps (any match)	<code>genre_ids && ARRAY[28,35]</code>
<code>(double pipe)</code>	Concatenate	<code>genre_ids + ARRAY[9648]</code>
<code>[n]</code>	Access element	<code>genre_ids[1] (1-indexed!)</code>

Querying Arrays

```
-- Find Action films
-- = ANY() checks if value exists anywhere in the array
SELECT f.title
FROM bluebox.film f
JOIN bluebox.film_genre fg ON fg.genre_id = ANY(f.genre_ids)
WHERE fg.name = 'Action'
LIMIT 3;
```

```
-- Find films that are both Action AND Crime
-- @> checks if array contains ALL specified values
SELECT title FROM bluebox.film f
WHERE genre_ids @> ARRAY(
    SELECT genre_id FROM bluebox.film_genre
    WHERE name IN ('Action', 'Crime')
) LIMIT 3;
```

```
-- Unnest: expand array to rows (list all genres for a film)
SELECT f.title, g.name as genre
FROM bluebox.film f, unnest(f.genre_ids) as gid
JOIN bluebox.film_genre g ON g.genre_id = gid
WHERE f.title = 'The Dark Knight';
```

JSON vs JSONB

Feature	JSON	JSONB
Storage	Text (as-is)	Binary (parsed)
Insert speed	Faster	Slower
Query speed	Slower	Much faster
Indexing	 No	 GIN indexes
Preserves order	 Yes	 No
Duplicate keys	Preserved	Last value wins

Always use JSONB unless you need exact JSON preservation

Working with JSONB

```
-- Create table with JSONB
CREATE TABLE movie_metadata (
    movie_id INT PRIMARY KEY,
    data JSONB
);

-- Insert JSON data
INSERT INTO movie_metadata (movie_id, data)
VALUES (1, '{
    "director": "Christopher Nolan",
    "budget": 160000000,
    "awards": ["Oscar", "BAFTA"]
}');
```

```
-- Different row, completely different structure - that's OK!
INSERT INTO movie_metadata (movie_id, data)
VALUES (2, '{
    "streaming": ["Netflix", "Hulu"],
    "runtime_minutes": 148,
    "has_sequel": true
}');
```

No schema enforcement -
each row can have different
structures!

JSONB Operators

Operator	Meaning	Example
<code>-></code>	Get as JSON	<code>data->'awards'</code>
<code>->></code>	Get as TEXT	<code>data->>'director'</code>
<code>#></code>	Get path as JSON	<code>data#>'{awards,0}'</code>
<code>#>></code>	Get path as TEXT	<code>data#>>'{awards,0}'</code>
<code>?</code>	Key exists?	<code>data ? 'budget'</code>
<code>?&</code>	All keys exist?	<code>data ?& array['a','b']</code>
<code>@></code>	Contains?	<code>data @> '{"x":1}'</code>

Querying JSONB

-- Extract value as text (most common)

```
SELECT data->>'director' as director  
FROM movie_metadata;
```

-- Extract nested value (awards array, first item)

```
SELECT data->'awards'->>0 as first_award  
FROM movie_metadata;
```

-- Check if key exists

```
SELECT * FROM movie_metadata  
WHERE data ? 'budget';
```

-- Query by JSON value (containment)

```
SELECT * FROM movie_metadata  
WHERE data @> '{"director": "Christopher Nolan"}';
```

Window Functions

Perform calculations across related rows **without grouping**

- **Running totals** - cumulative sums as you go down rows
- **LAG / LEAD** - access previous or next row's values
- **ROW_NUMBER** - assign sequential numbers to rows
- **RANK / DENSE_RANK** - rank rows with tie handling
- **PARTITION BY** - restart calculations for each group

Key difference from GROUP BY: window functions keep all rows!

Table Data

order_date	order_id	customer_id	total_amount
2024-08-21 10:00:00	21	1	349.98
2024-08-22 11:30:00	22	2	899.98
2024-08-23 09:15:00	23	3	34.98
2024-08-24 14:45:00	24	4	89.99
2024-08-25 08:25:00	25	5	149.99
2024-08-26 12:05:00	26	6	64.98
2024-08-27 16:35:00	27	7	249.98
2024-08-28 10:20:00	28	8	129.99
2024-08-29 13:10:00	29	9	179.98
2024-08-30 17:50:00	30	10	349.99
2024-08-31 11:00:00	31	1	199.98
2024-09-01 10:00:00	1	1	714.98
2024-09-01 10:00:00	11	1	714.98
2024-09-01 12:30:00	32	2	249.98
2024-09-02 10:15:00	33	3	19.99
2024-09-02 11:30:00	2	2	1199.99
2024-09-02 11:30:00	12	2	1199.99
2024-09-03 09:15:00	13	3	14.99
2024-09-03 09:15:00	3	3	14.99
2024-09-03 15:45:00	34	4	34.99

Frames

order_date	order_id	customer_id	total_amount
previous rows +			
2024-08-31 11:00:00	31	1	199.98

order_date	order_id	customer_id	total_amount
previous rows +			
2024-09-01 12:30:00	32	2	249.98

order_date	order_id	customer_id	total_amount
previous rows +			
2024-09-03 09:15:00	13	3	14.99

Window Function Output

running_total	order_date	order_id	customer_id	total_amount
2699.82	2024-08-31 11:00:00	31	1	199.98
4129.78	2024-09-01 10:00:00	1	1	714.98
4129.78	2024-09-01 10:00:00	11	1	714.98
4379.76	2024-09-01 12:30:00	32	2	249.98
4399.75	2024-09-02 10:15:00	33	3	19.99
6799.73	2024-09-02 11:30:00	2	2	1199.99
6799.73	2024-09-02 11:30:00	12	2	1199.99
6829.71	2024-09-03 09:15:00	13	3	14.99

Running Totals

```
SELECT
    payment_date::date,
    amount,
    SUM(amount) OVER (
        ORDER BY payment_date
    ) as running_total
FROM bluebox.payment
WHERE customer_id = 53853
ORDER BY payment_date;
```

payment_date	amount	running_total
2025-03-05	1.99	1.99
2025-06-07	7.96	9.95
2025-06-28	7.96	17.91

LAG - Compare to Previous Row

```
SELECT
    payment_date::date,
    amount,
    LAG(amount) OVER (ORDER BY payment_date) as prev_amount,
    amount - LAG(amount) OVER (ORDER BY payment_date) as difference
FROM bluebox.payment
WHERE customer_id = 53853
ORDER BY payment_date;
```

payment_date	amount	prev_amount	difference
2025-03-05	1.99	[NULL]	[NULL]
2025-06-07	7.96	1.99	5.97
2025-06-28	7.96	7.96	0.00

RANK with PARTITION BY

```
SELECT title, rating, vote_average,  
       RANK() OVER (PARTITION BY rating  
                    ORDER BY vote_average DESC) AS rank_in_rating  
FROM bluebox.film  
WHERE rating IN ('G', 'PG') AND vote_count > 18000  
ORDER BY rating, rank_in_rating;
```

title	rating	vote_average	rank_in_rating
Finding Nemo	G	7.8	1
Harry Potter and the Prisoner of Azkaban	PG	8	1
Up	PG	8	1
Harry Potter and the Philosopher's Stone	PG	7.9	3
Inside Out	PG	7.9	3
Harry Potter and the Half-Blood Prince	PG	7.7	5
Harry Potter and the Chamber of Secrets	PG	7.7	5

PARTITION BY rating restarts the ranking for each rating. Ties share a rank and RANK skips ahead (1, 1, 3); DENSE_RANK wouldn't skip, ROW_NUMBER ignores ties.

One More Technique: CTEs

You can't filter on a window function in WHERE. Wrap it in a **CTE** (Common Table Expression) and filter the result:

```
WITH ranked AS (  
    SELECT title, rating, vote_average,  
           RANK() OVER (PARTITION BY rating  
                        ORDER BY vote_average DESC) AS rnk  
    FROM bluebox.film  
    WHERE vote_count > 5000  
)  
SELECT rating, rnk, title, vote_average  
FROM ranked  
WHERE rnk <= 2 AND rating IN ('G', 'PG', 'PG-13')  
ORDER BY rating, rnk;
```

rating	rnk	title	vote_average
G	1	The Lion King	8.3
G	2	2001: A Space Odyssey	8.1
PG	1	Spider-Man: Into the Spider-Verse	8.4
PG	1	The Empire Strikes Back	8.4
PG-13	1	Forrest Gump	8.5
PG-13	1	The Lord of the Rings: The Return of the King	8.5
PG-13	1	The Dark Knight	8.5

The CTE names one step of the query, so the final SELECT reads top to bottom: "top 2 per rating".

Questions?

Next: Postgres DBA Basics Nobody Told You